

ÉTUDE DE DESIGN · MVP

Du POC au MVP :

Chatbot RAG Puls-Events

Rapport de gestion de projet

Data Engineer · Formation NLP & Bases vectorielles

Puls-Events · Août 2026

Responsable technique	Jérémy
Alternant	Data Engineer
Date	Août 2026
Durée projet	12 semaines
Stack principale	GCP · LangChain · Vertex AI · ChromaDB

01 Analyse et synthèse des besoins

Puls-Events propose une plateforme de découverte d'événements culturels personnalisée selon la localisation, la période et les préférences de chaque utilisateur. Suite au POC réalisé il y a un mois — qui a validé la faisabilité d'un moteur de recherche sémantique basé sur RAG — l'équipe souhaite maintenant passer à un MVP déployable en production. Après échange avec Jérémy (responsable technique), j'ai identifié quatre besoins concrets :

Mémoire conversationnelle	Retenir l'historique utilisateur pour personnaliser les échanges au fil du temps.
Contexte géographique	Adapter dynamiquement les réponses à la localisation de l'utilisateur.
Recherche web temps réel	Enrichir les réponses via un agent web (smolagents / Hugging Face).
Monitoring de performance	Mesurer la qualité des réponses, la latence et la satisfaction utilisateur.

02 Analyse et choix de la solution cloud

Pour héberger le MVP, j'ai comparé les trois grands fournisseurs cloud selon des critères directement liés au projet : support IA/NLP, base vectorielle disponible, facilité de déploiement et coût pour un volume de trafic MVP :

Critère	AWS	GCP	Azure
Services IA/NLP	Bedrock, SageMaker	Vertex AI, Gemini	Azure OpenAI
Base vectorielle	OpenSearch	Vertex AI Vector Search	Azure AI Search
Facilité déploiement	★★★	★★★★	★★★
Coût MVP	Moyen	Compétitif ✓	Élevé
Éco. open source	Bon	Excellent ✓	Moyen

Mon choix : GCP (Google Cloud Platform). Vertex AI couvre à la fois les embeddings et le LLM sans avoir à gérer plusieurs services distincts. L'intégration avec Hugging Face facilitera l'utilisation de smolagents. Cloud Run gère le scaling automatiquement, ce qui est idéal pour un MVP dont la charge est encore incertaine.

03 Architecture technique du MVP

L'architecture du MVP s'appuie sur celle du POC, avec trois ajouts majeurs : la mémoire conversationnelle, la géolocalisation et l'agent de recherche web. Elle se décompose en 4 couches :

1. Interface utilisateur

API REST exposée via FastAPI — intégrable dans la plateforme Puls-Events existante.

2. Moteur RAG

Embeddings : text-embedding-gecko (Vertex AI)
Base vectorielle : ChromaDB (MVP) → Vertex AI Vector Search (prod)
LLM : Gemini Pro via Vertex AI · Orchestration : LangChain

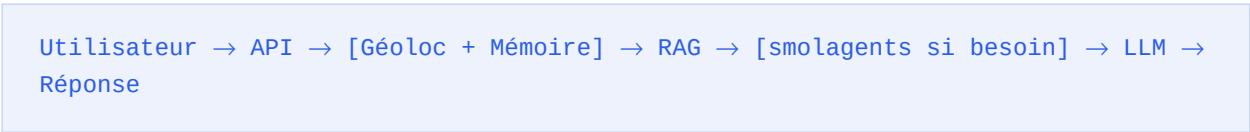
3. Modules complémentaires

Mémoire : historique par session stocké dans Firestore
Géolocalisation : filtre appliqué avant la recherche vectorielle
Recherche temps réel : agent smolagents déclenché si données insuffisantes

4. Monitoring

Langfuse : traçabilité des appels LLM, latence, qualité
Métriques custom : taux de satisfaction, pertinence des réponses

Flux d'une requête :



04 Macro backlog

J'ai listé les fonctionnalités à développer et les ai priorisées avec le modèle MoSCoW (Must / Should / Could / Won't). Les Must correspondent au strict minimum pour que le MVP soit utilisable ; les Should sont importants mais non bloquants au lancement :

Priorité	Epic	Fonctionnalité
■ Must	RAG Core	Indexation des événements Open Agenda
■ Must	RAG Core	Recherche sémantique vectorielle
■ Must	RAG Core	Génération de réponses via LLM
■ Must	Mémoire	Historique de conversation par utilisateur
■ Must	Géolocalisation	Filtrage des résultats par localisation
■ Should	Recherche web	Intégration agent smolagents
■ Should	Monitoring	Dashboard Langfuse (latence, qualité)
■ Should	API	Endpoint REST documenté (Swagger)
■ Could	UX	Interface chat intégrée à Puls-Events
■ Could	Monitoring	Score de satisfaction utilisateur
■ Won't	Multilingue	Support d'autres langues que le français

05 Estimation des coûts

Build — Développement et déploiement initial

Poste	Détail	Coût estimé
Développement	3 mois alternant (coût entreprise)	~3 000 €
Infrastructure setup	GCP projet, CI/CD, environnements	~200 €
APIs & outils	Vertex AI free tier, Langfuse open source	~0 €
Total build		~3 200 €

OPEX — Exploitation mensuelle (hypothèse : 1 000 users, 50 req/user/mois)

Service	Usage estimé	Coût/mois
Vertex AI (Gemini Pro)	50K requêtes × ~800 tokens	~70 €
Vertex AI (Embeddings)	50K requêtes × ~500 tokens	~3 €
Cloud Run	Serverless, faible charge MVP	~20 €
Firestore (mémoire)	< 1 GB	~5 €
Smolagents / recherche web	API calls limités	~10 €
Total OPEX		~108 €/mois

Ces estimations sont basées sur une hypothèse de 1 000 utilisateurs actifs. Si l'usage décolle rapidement (x10), l'OPEX monterait autour de 800 €/mois — à anticiper dans le budget dès la phase de croissance.

06 **Plan de projet**

Le projet est découpé en 6 phases sur **12 semaines**. J'ai prévu des jalons de validation avec Jérémy à la fin des phases clés pour s'assurer qu'on reste alignés avant de continuer.

Phase	Semaines	Activités	Livrable
1. Cadrage	S1–S2	Architecture, setup GCP, choix stack	Doc technique validée
2. RAG Core	S3–S5	Indexation, recherche vectorielle, LLM	Chatbot v1 fonctionnel
3. Enrichissement	S6–S8	Mémoire, géoloc, smolagents	Chatbot v2 complet
4. Monitoring	S9–S10	Langfuse, métriques, dashboard	Monitoring opérationnel
5. Tests	S11	Tests de charge, corrections, optimisation	Rapport de tests
6. Livraison MVP	S12	Déploiement prod, documentation	MVP en production

Jalons clés

Jalon	Description
J1 — S2	Architecture validée par Jérémy

J2 — S5	Démo RAG Core (chatbot de base)
J3 — S8	Démo MVP complet (toutes fonctionnalités)
J4 — S12	Livraison production + documentation